Write pseudocode for multiple tasks safely sharing data using read-write semaphores.

1. Problem Understanding

Multiple tasks may need to access a shared data resource.

- Several tasks can READ the data simultaneously.
- Only one task can WRITE the data and writers must have exclusive access.
- This requires synchronization to avoid data inconsistency.

Given :

- Shared resource: SharedData
- Multiple Reader tasks: R1, R2, …, Rn
- Writer task(s): W
- Use Read-Write Semaphore (RW_Sem)

2. Suitable RTOS / Feature

RTOS : FreeRTOS / Any RTOS

Feature : Read-Write Semaphore (configUSE_RW_SEMAPHORE = 1)

Why Read-Write Semaphore?

- Allows multiple readers concurrently.
- Ensures exclusive access for writers.
- Prevents data inconsistency.
- Improves system concurrency and performance.

3. Structured Pseudocode – Initialization

```
// Shared Resource Declaration
TYPE SharedData
    INTEGER temperature
    INTEGER pressure
    INTEGER humidity
END_TYPE

// Read-Write Semaphore Handle
RW_SEMAPHORE RW_Sem

// System Initialization
FUNCTION System_Init()
    CREATE_RW_SEMAPHORE(RW_Sem)
    INITIALIZE SharedData
END_FUNCTION
```

4. Reader Task(s) – Pseudocode

```
TASK Reader_Task (id)
BEGIN
 LOOP
   // Request access to read
   RW_Sem_TakeRead(RW_Sem)  // Wait if writer active
   // ---- Critical Section (Read) -----
   READ SharedData.temperature
   READ SharedData.pressure
   READ SharedData.humidity
   // ---- End Critical Section -----
   RW_Sem_GiveRead(RW_Sem)  // Release read lock
   PROCESS Data (Display / Log / Transmit)
   DELAY Some_Time
 END LOOP
END TASK
```

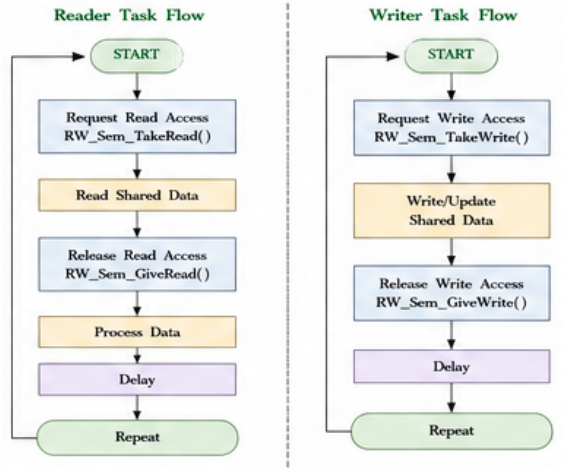
5. Writer Task – Pseudocode

```
TASK Writer_Task
BEGIN
 LOOP
   // Request exclusive access to write
   RW_Sem_TakeWrite(RW_Sem)  // Wait if readers/writers
   // ---- Critical Section (Write) -----
   UPDATE SharedData.temperature
   UPDATE SharedData.pressure
   UPDATE SharedData.humidity
   // ---- End Critical Section -----
   RW_Sem_GiveWrite(RW_Sem)  // Release write lock
   DELAY Some_Time
 END LOOP
END TASK
```

6. Read-Write Semaphore Operations Summary

Operation	Used By	Purpose	Behavior
RW_Sem_TakeRead( )	Reader Task	Request read access	Blocks if a writer is active
RW_Sem_GiveRead( )	Reader Task	Release read access	Allows waiting writers if any
RW_Sem_TakeWrite( )	Writer Task	Request write access	Blocks if any reader or writer active
RW_Sem_GiveWrite( )	Writer Task	Release write access	Allows waiting readers/writers

7. Logical Flow Diagram



8. Real-Time Example – Hospital Patient Monitoring System

In a hospital ICU monitoring system:

- Multiple display tasks (R1, R2, R3) read patient vital signs (Heart Rate, BP, SpO2, Temp) to show on different monitors.
- One logging task (W) updates the vital signs into the shared database.
- Read-Write Semaphore is used to ensure that while data is being updated by the logging task, no other task reads partial or inconsistent data.

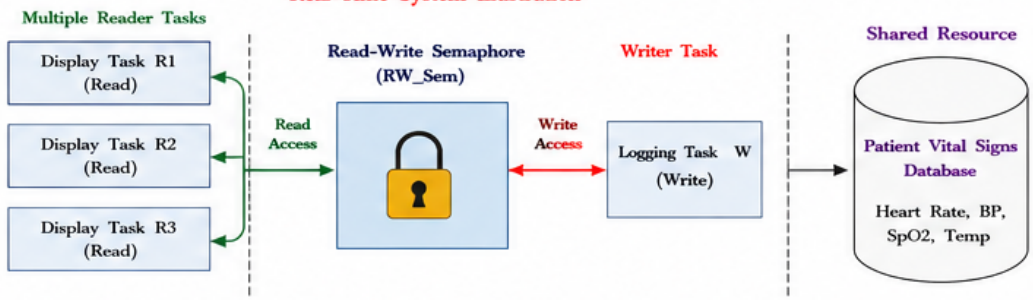
9. Key Points

- Multiple readers can access the data simultaneously.
- Only one writer gets exclusive access.
- Writers are given priority once they start waiting (to avoid writer starvation).
- Increases system concurrency and data consistency.
- Suitable for database caching, file systems, sensor data sharing, configuration tables, etc.

10. Assumptions

- RW_Semaphore supports priority inheritance (if RTOS provides).
- All tasks run under preemptive scheduler.
- No task holds the semaphore longer than necessary.
- Context switch occurs when higher priority task is ready.
- Deadlock avoided by proper take/give sequence.

Real-Time System Illustration



Write pseudocode for two tasks acquiring mutexes using proper lock ordering to avoid deadlock.

1. Problem Statement

Two tasks T1 and T2 need to acquire two mutexes M1 and M2 to access shared resources.

Show pseudocode using proper lock ordering to avoid deadlock.

Given :

- Two tasks: T1 and T2
- Two mutexes: M1 and M2
- Both tasks need both mutexes (in some order)
- Deadlock can occur if lock order is not consistent.

2. Suitable RTOS / Feature

RTOS : FreeRTOS / Any RTOS

Feature : Mutex with Priority Inheritance (configUSE_MUTEXES = 1)

Why Lock Ordering?

- If tasks lock mutexes in different order, deadlock can occur.
- If all tasks follow a fixed global order (e.g., M1 before M2), deadlock is avoided.
- Ensures system reliability.

3. Lock Ordering Rule

Always lock mutexes in the same global order:

First M1, then M2

Rule:

- All tasks must acquire mutexes in the same predefined order.
- Never acquire a mutex with a lower priority order while holding a higher order mutex.
- Release in reverse order.

4. Task T1 – Pseudocode

```
TASK T1
BEGIN
  LOOP
    // Acquire mutexes in global order: M1 then M2
    lock(M1);       // Lock M1
    lock(M2);       // Lock M2

    // ---- Critical Section ----
    // Access shared resources protected by M1 and M2
    perform_Task2_operations();
    // ---- End Critical Section ----
    unlock(M2);     // Release in reverse order
    unlock(M1);
    DELAY(some_time);
  END LOOP
END TASK
```

5. Task T2 – Pseudocode

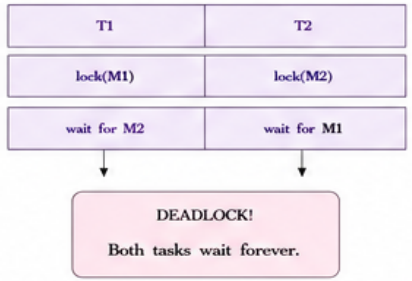
```
TASK T2
BEGIN
  LOOP
    // Acquire mutexes in the same global order: M1 then M2
    lock(M1);       // Lock M1
    lock(M2);       // Lock M2

    // ---- Critical Section ----
    // Access shared resources protected by M1 and M2
    perform_Task2_operations();
    // ---- End Critical Section ----
    unlock(M2);     // Release in reverse order
    unlock(M1);
    DELAY(some_time);
  END LOOP
END TASK
```

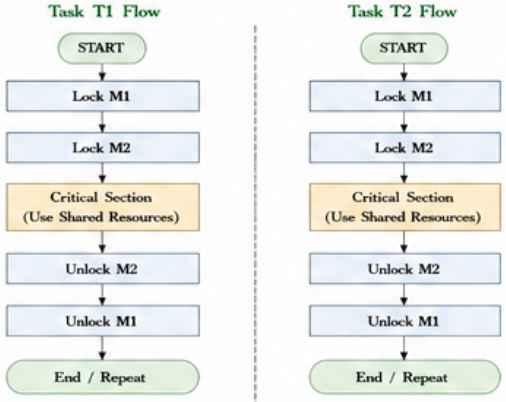
Key Rule : All tasks must acquire mutexes in the same order (M1 → M2) and release in reverse order to avoid deadlock.

6. What if Wrong Order is Used?

If T1 locks M1 then M2, but T2 locks M2 then M1, deadlock can occur.



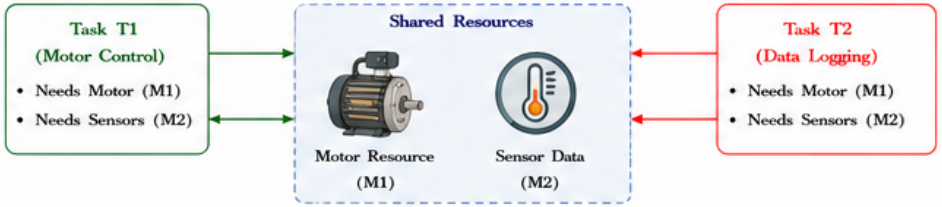
7. Logical Flow Diagram



8. Real-Time Example – Industrial Control System

Example: Two tasks controlling different parts of a machine.

- Task T1 (Motor Control) needs access to Motor Resource (M1) and Sensor Data (M2).
- Task T2 (Data Logging) also needs both resources.
- Using the same lock order (M1 → M2) prevents deadlock.



Both tasks always lock M1 first, then M2. Hence, deadlock is avoided.

9. Key Points

- Always follow a global lock order.
- Never acquire a lower-order mutex while holding a higher-order mutex.
- Release mutexes in reverse order of acquisition.
- Prevents circular wait condition (one of the four necessary conditions for deadlock).
- Improves system reliability and predictability.

10. Assumptions

- Mutexes M1 and M2 are independent.
- lock() blocks if the mutex is not available.
- unlock() releases the mutex.
- Tasks are periodic or event-driven.
- No mutex is held longer than necessary.

11. Summary

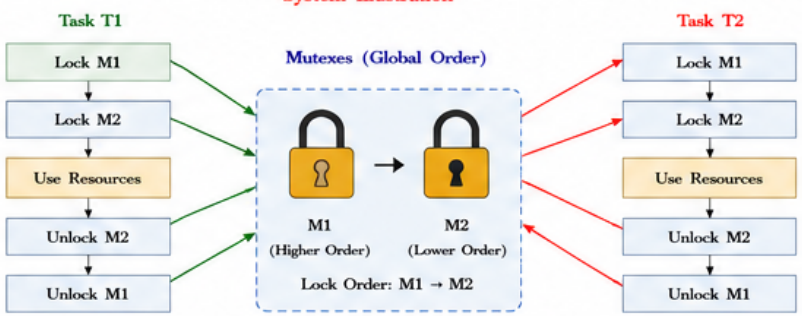
By enforcing a consistent global lock ordering (M1 → M2) across all tasks, we eliminate the possibility of circular wait, thus preventing deadlock and ensuring safe access to shared resources.

Global Lock Order:

Always lock M1 first, then M2

Always unlock M2 first, then M1

System Illustration



Write pseudocode to detect and recover from deadlock in a system sharing multiple resources.

1. Problem Statement

In a system where multiple processes share multiple resources, deadlock may occur.

Write pseudocode to:

- Detect the occurrence of deadlock.
- Recover from deadlock.

Given :

- n processes: P0, P1, …, Pn−1
- m resource types: R1, R2, …, Rm
- Allocation, Max, Available matrices

2. Suitable Approach / Feature

Approach : Deadlock Detection using Banker's Algorithm

Feature : Periodically check for deadlock and recover by aborting one of the deadlocked processes.

Why Detection & Recovery?

- Deadlock can stop system progress.
- Detection finds deadlock in time.
- Recovery releases resources and restores normal operation.

3. Data Structures

MATRICES:

- Available[m] : Available instances of each resource
- Max[n][m] : Maximum demand of each process
- Allocation[n][m] : Resources currently allocated
- Need[n][m] : Remaining need (Max - Allocation)

WHERE:

Need[i][j] = Max[i][j] - Allocation[i][j]

4. Pseudocode – Deadlock Detection

```
FUNCTION Detect_Deadlock(Available, Allocation, Max, n, m)
    FOR i ← 0 TO n−1
        FOR j ← 0 TO m−1
            Need[i][j] ← Max[i][j] - Allocation[i][j]
        END FOR
    END FOR

    Work[j] ← Available[j]    FOR all j
    Finish[i] ← false         FOR all i

 REPEAT
    found ← false
    FOR i ← 0 TO n−1
        IF Finish[i] == false AND Need[i] ≤ Work THEN
            FOR j ← 0 TO m−1
                Work[j] ← Work[j] + Allocation[i][j]
            END FOR
            Finish[i] ← true
            found ← true
        END IF
    END FOR
 UNTIL found == false

 DEADLOCKED ← empty list
 FOR i ← 0 TO n−1
    IF Finish[i] == false THEN
        add P_i to DEADLOCKED
    END IF
 END FOR
 RETURN DEADLOCKED
END FUNCTION
```

5. Pseudocode – Deadlock Recovery

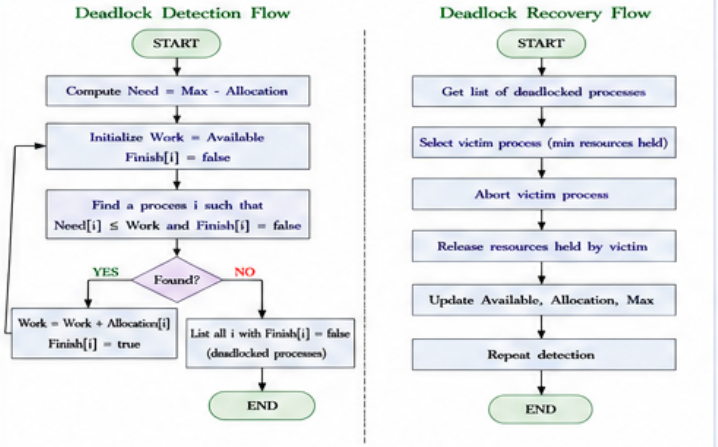
```
FUNCTION Recover_Deadlock(DEADLOCKED)
    IF DEADLOCKED is empty THEN
        RETURN     // No deadlock
    END IF

    // Recovery Strategy: Abort one process at a time
    // (Choose the process with minimum total resources allocated)

    victim ← Select_Victim(DEADLOCKED)
    Abort_Process(victim)
    // Release resources held by victim
    FOR j ← 0 TO m−1
        Available[j] ← Available[j] + Allocation[victim][j]
        Allocation[victim][j] ← 0
        Max[victim][j] ← 0
    END FOR
    Remove victim from DEADLOCKED
    RETURN
END FUNCTION

FUNCTION Select_Victim(DEADLOCKED)
    min_resources ← ∞, victim ← −1
    FOR each p in DEADLOCKED
        total ← SUM over j (Allocation[p][j])
        IF total < min_resources THEN
            min_resources ← total
            victim ← p
        END IF
    END FOR
    RETURN victim
END FUNCTION
```

6. Logical Flow Diagram



7. Real-Time Example – Operating System

Example: 5 Processes (P0–P4) and 3 Resource Types (R1, R2, R3)

Initial State:

	Allocation		
	R1	R2	R3
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Max		
R1	R2	R3
7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

Available		
R1	R2	R3
3	3	2

Need (Max - Allocation)		
R1	R2	R3
P0	4	3
P1	1	2
P2	6	0
P3	0	1
P4	4	1

Detection Result:

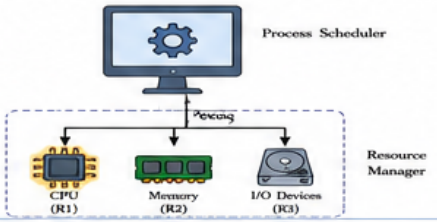
Deadlocked Processes = { P1, P3, P4 }

Recovery Step:

Abort P1 (holds min resources = 2). Release its resources (2,0,0).

New Available = (3,3,2) + (2,0,0) = (5,3,2)

Run detection again → System becomes safe.



8. Key Points

- Deadlock occurs when processes wait indefinitely for resources.
- Banker's algorithm is used for deadlock detection.
- If deadlock is detected, recovery is done by aborting one or more processes.
- Aborting a process releases its resources.
- After recovery, run detection again to ensure the system is safe.
- Proper detection and recovery ensure system reliability and performance.

9. Assumptions

- Each resource type has a finite number of instances.
- Processes declare maximum demand in advance.
- Resources are allocated only if the system remains safe.
- Cost of process abort is acceptable.
- Detection is invoked periodically or on request.

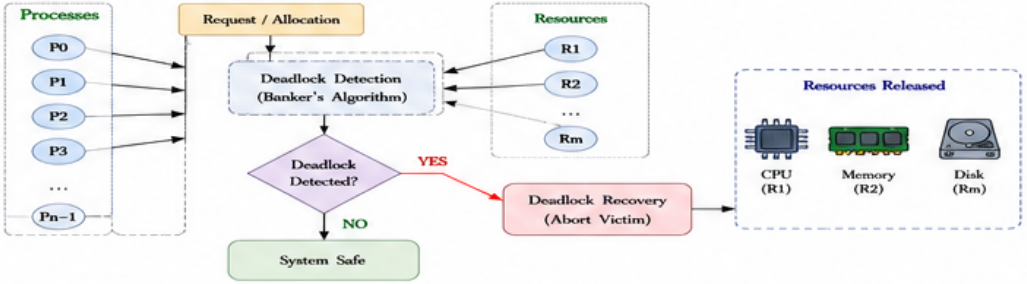
10. Summary

The pseudocode first detects deadlock using Banker's algorithm by checking if all processes can finish. If some processes cannot finish, they are in deadlock. The recovery procedure aborts one deadlocked process (victim) to release its resources and make them available. Then the system repeats detection to restore normal operation.

Algorithm Steps:

1. Compute Need matrix
2. Detect deadlocked processes
3. Abort a victim process
4. Release its resources
5. Repeat detection until no deadlock

11. System Illustration



Write pseudocode for resource allocation in an industrial plant while avoiding deadlock and starvation.

1. Problem Statement

In an industrial plant, multiple processes (tasks) request different types of resources (machines, tools, operators, energy, etc.).

Write pseudocode for resource allocation such that:

- Deadlock is avoided.
- Starvation is prevented.
- Resources are allocated efficiently and fairly.

Given :

- n processes: P1, P2, …, Pn
- m resource types: R1, R2, …, Rm
- Max, Allocation, Available matrices

2. Suitable Approach / Feature

Approach :

Banker's Algorithm + Fair Scheduling (FIFO Queue)

Features :

- Banker's algorithm ensures the system never enters an unsafe state (avoid deadlock).
- FIFO waiting queue ensures fair access and prevents starvation.
- Resources are allocated only if the request keeps the system in a safe state.

Why this works?

- Safety check avoids deadlock.
- FIFO queue gives every waiting process a chance → avoids starvation.

3. Data Structures

MATRICES :

- Available[m] : Available instances of each resource
- Max[n][m] : Maximum demand of each process
- Allocation[n][m] : Resources currently allocated
- Need[n][m] : Remaining need = Max − Allocation

OTHER STRUCTURES :

- Queue Q : FIFO queue of processes waiting for resources (to ensure fairness)
- bool Finish[n] : true if process has completed

WHERE :

Need[i][j] = Max[i][j] − Allocation[i][j]

4. Pseudocode – Resource Request & Allocation

```
FUNCTION Request_Resource(i, Request[i][m])
  // i → process making request
  // Request[i][j] → instances of Rj requested by Pi
  IF Request[i][j] ≤ Need[i][j]  FOR all j
   AND Request[i][j] ≤ Available[j]  FOR all j  THEN
    // Pretend allocation
    FOR j ← 0 TO m-1 DO
      Available[j]  ←  Available[j] - Request[i][j]
      Allocation[i][j] ← Allocation[i][j] + Request[i][j]
      Need[i][j] ← Need[i][j] - Request[i][j]
    END FOR
    IF Safety_Check() = TRUE THEN
      // Allocation safe
      Dequeue(Q)      // remove i from waiting queue if present
      RETURN GRANTED
    ELSE
      // Not safe → rollback
      FOR j ← 0 TO m-1 DO
        Available[j]  ←  Available[j] + Request[i][j]
        Allocation[i][j] ← Allocation[i][j] - Request[i][j]
        Need[i][j] ← Need[i][j] + Request[i][j]
      END FOR
      Enqueue(Q, i)      // keep process in queue
      RETURN WAIT
    ENE IF
  ELSE
    Enqueue(Q, i)      // not enough resources or exceeds need
    RETURN WAIT
  END IF
END FUNCTION
```

5. Pseudocode – Safety Check (Banker's Algorithm)

```
FUNCTION Safety_Check()
  Work[m]  ←  Available
  Finish[n]  ←  false for all

  WHILE ∃ i such that Finish[i] = false
       AND Need[i] ≤ Work (for all j)   DO
     FOR j ← 0 TO m-1 DO
         Work[j]  ←  Work[j] + Allocation[i][j]
     END FOR
     Finish[i]  ←  true
  END WHILE

  IF Finish[i] = true FOR all i THEN
     RETURN TRUE      // safe state
  ELSE
     RETURN FALSE      // unsafe state
  END IF
END FUNCTION
```

6. Pseudocode – Resource Release

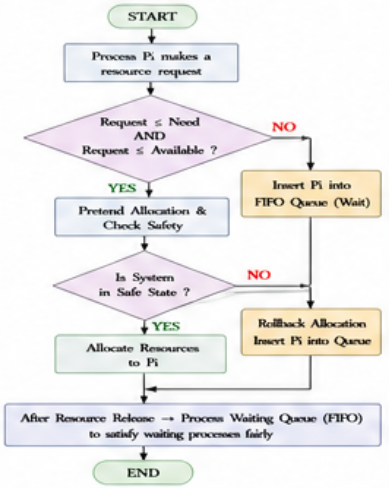
```
FUNCTION Release_Resource(i, Release[i][m])
  FOR j ← 0 TO m-1 DO
     Allocation[i][j] ← Allocation[i][j] - Release[i][j]
     Need[i][j]  ←  Need[i][j] + Release[i][j]
     Available[j]  ←  Available[j] + Release[i][j]
  END FOR
  Finish[i]  ←  true
  // After release, try to satisfy waiting processes fairly
  Call Process_Waiting_Queue()
END FUNCTION
```

7. Pseudocode – Process Waiting Queue (Fairness)

```
FUNCTION Process_Waiting_Queue()
   // Try processes in FIFO order to avoid starvation
   temp ← size(Q)
   REPEAT temp times
      i ← Front(Q)          // peek oldest waiting process
      IF  Safety_Check_After_Request(i) = TRUE THEN
         Dequeue(Q)
         Allocate resources to i (as in Request_Resource success part)
      ELSE
         // Move to rear to give chance to others
         Dequeue(Q)
         Enqueue(Q, i)
      END REPEAT
END FUNCTION

FUNCTION Safety_Check_After_Request(i)
  // Same as in Request_Resource pretend & check part
  // Return TRUE if safe, else FALSE
END FUNCTION
```

8. Logical Flow Diagram



9. Real-Time Example – Industrial Plant

Resources in Plant :

R1 – CNC Machine R2 – Welding Robot R3 – Material Handler R4 – Operator

Processes (Tasks) :

P1 – Cutting Job P2 – Assembly Job P3 – Welding Job P4 – Painting Job
P5 – Quality Check

Goal : Allocate machines and operators safely without deadlock, and ensure every job eventually gets resources (no starvation).

Initial State Example

Process	Max				Allocation				Need (Max − Allocation)			
	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4
P1	1	1	1	1	0	1	0	0	1	0	1	1
P2	1	0	1	1	1	0	1	0	0	0	0	1
P3	0	1	1	0	0	1	0	0	0	0	1	0
P4	1	1	0	1	1	0	0	1	0	1	0	0
P5	0	0	1	1	0	0	0	0	0	0	1	1
Available	1	0	1	1								

How it works in plant :

- Each job requests required machines/operators.
- System grants request only if it keeps system safe.
- If not safe, job waits in FIFO queue.
- When a job finishes and releases resources, waiting jobs are checked in queue order.

10. Key Points

- Banker's algorithm ensures no deadlock (safe state only).
- FIFO waiting queue ensures fairness → no starvation.
- Resources are allocated only if request ≤ need and available and system remains safe.
- After release, waiting processes are retried in queue order.
- This ensures efficient, safe and fair resource allocation.

11. Assumptions

- Maximum demand (Max) of each process is known.
- All resources are reusable and independent.
- Processes make requests one at a time.
- Resources are released after process completion.
- Context switching and delays are negligible.

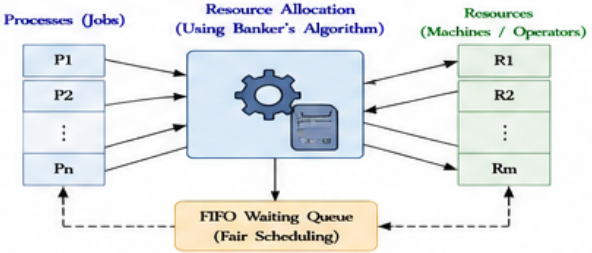
12. Summary

By combining Banker's algorithm (safety) with FIFO queue (fairness), we can allocate resources in an industrial plant while avoiding deadlock and starvation, ensuring high reliability and productivity.

Algorithm Steps :

1. Check request validity
2. Pretend allocation
3. Safety check (Banker's Algorithm)
4. Allocate if safe else wait (FIFO)
5. On release → update & process queue
6. Repeat

13. System Illustration



Write pseudocode for sending sensor data to a logging task using a message queue.

1. Problem Statement

A sensor task periodically reads data from a sensor and sends it to a logging task. The logging task receives the data from a message queue and logs it (e.g., to SD card or console file).

Goal :

Transfer sensor data safely between tasks using a message queue.

Given :

- Sensor Task (Producer)
- Logging Task (Consumer)
- Message Queue (MQ)
- RTOS with queue support

2. Suitable RTOS / Feature

RTOS : FreeRTOS / Any RTOS

Feature : Message Queue (configUSE_QUEUE_SETS = 1) optional)

Why Message Queue?

- Safe data transfer between tasks.
- Decouples producer and consumer.
- Prevents data loss and race conditions.
- Queue handles synchronization internally.

3. Data Structures

```
#define QUEUE_LENGTH   10
   // number of messages

typedef struct {
   uint32_t timestamp;    // time in ms
   float temperature;     // sensor value 1
   float humidity;        // sensor value 2
   uint16_t status;       // status / error code
} sensor_msg_t;


// Message Queue Handle

QueueHandle_t    sensorQueue;
```

4. Pseudocode – Sensor Task (Producer)

```
TASK SensorTask
BEGIN
   sensor_msg_t  msg;
 LOOP
   // Read sensor values
   msg.timestamp   = GetSystemTime();
   msg.temperature = Read_Temperature();
   msg.humidity    = Read_Humidity();
   msg.status      = Read_Status();

   // Send data to queue (block if queue full)
   xQueueSend(sensorQueue, &msg, portMAX_DELAY);

   // Wait until next sample period
   vTaskDelay(pdMS_TO_TICKS(1000));  // 1 sec
 END LOOP
END TASK
```

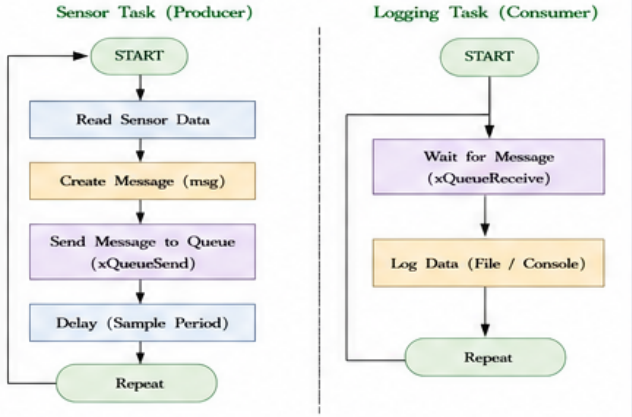
5. Pseudocode – Logging Task (Consumer)

```
TASK LoggingTask
BEGIN
   sensor_msg_t  msg;
 LOOP
   // Wait to receive data from queue
   if (xQueueReceive(sensorQueue, &msg, portMAX_DELAY) == pdPASS) then
      // Log the received data
      LogToFile(msg);        // e.g., write to SD card or file
      // Or print to console
      // printf("%lu, %.2f, %.2f, %d\n", msg.timestamp,
      //          msg.temperature, msg.humidity, msg.status);
   end if
 END LOOP
END TASK
```

6. Queue Operations Summary

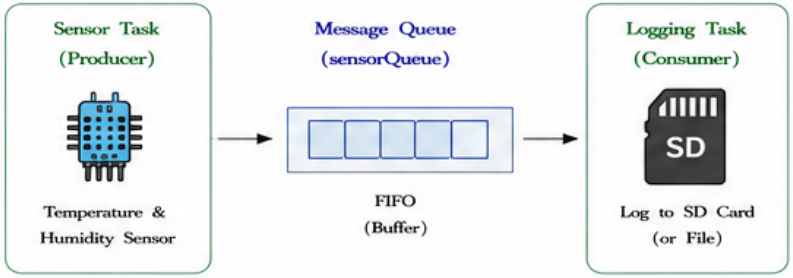
Operation	Used By	Purpose	Behavior
xQueueSend()	Sensor Task	Send sensor data to queue	Blocks if queue is full (portMAX_DELAY)
xQueueReceive()	Logging Task	Receive data from queue	Blocks if queue is empty (portMAX_DELAY)
Queue Length	Both	Limited buffer size	Prevents memory overflow
FIFO	Queue	First In First Out order	Maintains order of sensor readings

7. Logical Flow Diagram



8. Real-Time Example – Environmental Monitoring System

An environmental monitoring node reads temperature and humidity from a sensor every second and sends the data to a logging task, which stores it in an SD card.



9. Key Points

- Sensor task acts as producer and sends messages.
- Logging task acts as consumer and receives messages.
- Message queue provides safe and synchronized data transfer.
- FIFO order ensures readings are logged in sequence.
- Blocking send/receive prevents data loss and racing.
- Queue length must be chosen according to memory and data rate.

10. Assumptions

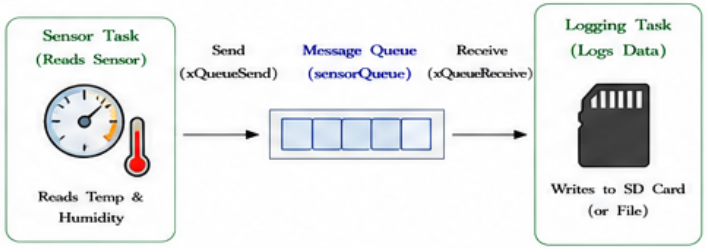
- RTOS supports message queues (e.g., FreeRTOS).
- Sensor read functions are non-blocking and fast.
- Logging (file/SD write) is faster than data rate.
- Enough stack and heap are available.
- Queue size is sufficient for burst data.

11. System Initialization (Pseudocode)

```
FUNCTION System_Init()
BEGIN
   // Create message queue
   sensorQueue = xQueueCreate(QUEUE_LENGTH,
                              sizeof(sensor_msg_t));
   if(sensorQueue == NULL) then
      Error_Handler();
   end if

   // Create tasks
   xTaskCreate(SensorTask,  "SensorTask",  256, NULL, 2, NULL);
   xTaskCreate(LoggingTask, "LoggingTask", 512, NULL, 1, NULL);
END FUNCTION
```

12. System Illustration



Summary : SensorTask reads data and sends to Message Queue. LoggingTask receives and logs it. Message queue ensures safe, ordered, and synchronized communication between tasks.